Chanakya An AI teacher that works offline: narrated, animated lessons in nine languages

Team name: The Last Commit · **Members:** Aryan Dubey, Priyadarshini Prasad, Niharika K P, Dhiraj Shetty
College: RV University, Bengaluru · **Course:** B.Tech (Hons) · **Branch:** Computer Science Engineering · **Year:** 2nd year
Repo: github.com/signingoff-dubey/UNLOX-TEACHER · **Version:** v0.16.0 · **Built for:** the Unlox hackathon

Vision

Quality explanation should not depend on a fast connection, a subscription or a particular language. Chanakya turns any topic into a complete video lesson on the learner's own device: type a question, or attach a photo or a PDF, and it plans the lesson, narrates it in the learner's language, animates it and lets the learner watch it, question it and take it away as an MP4 with captions. Because every model runs locally and no outside AI service is ever called, a lesson costs nothing to produce, works without internet, and keeps the learner's questions, photos and documents private. The same product also runs natively on an Android phone.

How it works

The design rests on one principle: the AI only returns *data*. A language model writes a JSON lesson that is validated against a schema, choosing one of eleven scene templates and filling in its text, while our own engine draws, times and renders every frame; nothing the model writes is ever executed. A request is first cleaned into its subject ("teach me about dsa in java" becomes "DSA in Java"), checked against what the learner already owns, and grounded in an offline knowledge base of encyclopedia articles, curated reference notes and any document the learner attached. The plan is then refined by a pipeline of specialists: a quality gate that rejects repetition and thin narration, a visual director that chooses the right scene for each idea, a writer that enriches narration, and a fact checker that verifies the statements. The approved English plan is translated, each sentence is spoken with a delivery that suits it, and every scene lasts exactly as long as its narration, so voice, captions and animation stay in sync.

Capabilities

Lessons range from 30 seconds to 4 minutes, and a topic can become a course of one to three videos. The eleven scene types are title, bullets, flow, cycle, compare, diagram, quiz, summary, **graph** (lines, parabolas, circles, ellipses, waves and more, with points, tangents and chords), **code** (syntax-coloured, with a highlight that follows the narration) and **network** (boxes and arrows for trees, linked structures and architectures). Four modes fit the purpose: Explanation, Study, Homework, which teaches the method and writes a separate answers file, and Revise, which re-teaches what the learner keeps missing. Photos of a page or a board are read by a local vision model, and PDFs of up to 2 GB are read as structured Markdown, with OCR for scans and descriptions of the pictures inside, so a lesson can teach a single chapter with page citations. Around each video sit a voice-enabled tutor that can build "one more video" on request, flashcards, quizzes, mind maps, glossaries, spaced-repetition review, teach-it-back grading, rewards, worksheets and question papers, MP4, caption and audio export, an offline bundle that can be shared on a USB stick, and Learnverse, a walkable 3D world generated from any topic.

Flash and Pro

Learners choose how a video is built. **Flash** starts playback within seconds and keeps writing while the learner watches, each part knowing what the earlier parts have taught. **Pro** plans the whole video before showing it: it outlines every scene, writes the first minute in one pass and patches each further minute on with the full outline and all earlier text in view, then gives the fact checker extra time. The result is a more coherent, carefully verified video, built with the same model as Flash.

Models used

All models run locally. The **language model** behind planning, directing, writing, fact checking, the tutor and the study tools is **Qwen2.5-7B-Instruct** (Q4_K_M, Apache-2.0), served by Ollama. **LLaVA 7B** reads photos and the pictures inside PDFs. Retrieval combines **nomic-embed-text** for the knowledge base, **multilingual-e5-small** for document passages and **bge-reranker-v2-m3** for re-ranking. Translation uses **NLLB-200** (distilled 600M), with **AI4Bharat IndicTrans2** as an alternative and a back-translation check on every sentence. Narration uses **Piper** neural voices, **AI4Bharat Indic-TTS** voices and **Indic Parler-TTS** for expressive speech. Spoken questions are understood by **faster-whisper**, and scanned pages are read by **Tesseract** and **Unlimited-OCR**. On Android the same roles are filled by **Gemma 4 E2B** (LiteRT-LM), Piper through **sherpa-onnx**, **Whisper** and **Vosk**, and **ML Kit** for translation and text recognition.

Languages and voices

Chanakya teaches in English, Hindi, Telugu, Marathi, Bengali, Malayalam, Tamil, Kannada and Gujarati, each with a natural voice and correctly rendered script, and the language list is configuration-driven so it can grow. Lesson length is balanced to each voice's speaking pace, voices fall back gracefully between engines, and every translation is verified for script and meaning before it is shown.

Accuracy and trust

A lesson is only as good as its facts, so every plan is checked before it is spoken. The fact checker works entirely on the device: it calculates arithmetic and equations, verifies counts and well-known lists such as the colours of the rainbow and the order of the planets, and compares each statement with passages retrieved from the knowledge base. A finding only counts when it quotes words that really appear in the source and a second, focused question agrees, and any correction must pass the same check. A **memory** of the learner's earlier videos lets new lessons build on them instead of repeating them, and technical subjects are taught with real code in the language the learner names or with a drawn diagram rather than analogy alone. Each lesson lists its sources and carries a report of what was verified or corrected.

Reliability

Jobs resume after interruption, heavy models share the hardware by taking turns, a graceful fallback keeps a course going if a step runs short of memory, requests to the model are retried automatically, and a built-in readiness check names anything missing and the command that fixes it. The backend is covered by 345 automated tests.

Benchmarks

Every figure below was measured end to end on one consumer laptop, with all models running locally and warm. **Responsiveness:** the time to first token is about 0.3 seconds, the first complete scene of a lesson is written in about 1.2 seconds (it was 3.2 seconds before the prompt was tailored to the subject), and the model decodes at roughly 43 tokens per second. In Flash mode the first scenes are watchable 6 to 18 seconds after the learner presses Create. **Video generation:** a one-minute lesson is fully built, meaning planned, checked, translated and narrated, in 40 to 70 seconds; a two-minute lesson takes two to three minutes including fact checking (about 133 seconds in Flash and 167 in Pro on a technical topic such as AVL trees); the narration of a three-minute lesson is synthesised in 10 to 17 seconds; and a three-minute MP4 at 720p renders in about 3.3 minutes. **Supporting steps:** knowledge-base search takes 35 to 60 milliseconds, translating 32 sentences takes 2.9 seconds, a repeated spoken question is answered in 0.29 seconds against about 10 seconds cold, a photo is accepted in 0.05 seconds so the Create button never waits, and a 1,000-page scanned PDF becomes searchable in under three minutes, with text-layer pages read in 1 to 15 milliseconds and converted to Markdown in 50 to 500 milliseconds.

Approach to speed

We treated latency as a design requirement and measured every change on the real models. First, we **avoid constrained decoding**: the schema is given to the model as text and validators repair small slips, which produced the same lesson plan in about 5 seconds instead of 12 to 22. Second, **prompts are built for the subject**: a general topic needs roughly 790 tokens of instructions instead of 1,700, which cut prompt processing from 0.95 to 0.30 seconds, while maths, code and diagram topics receive exactly the templates they need. Third, the plan is **streamed and parsed scene by scene**, so the player opens on the first finished scene while the rest is still being written. Fourth, we **overlap work**: finished scenes are translated while the planner is still writing, translations are batched by length (about 35% faster), and translations, narration clips and spoken-question transcripts are cached. Fifth, **models stay resident while in use and are released when idle**, a GPU slot lets the language model, translator and voices take turns, and a memory gate waits instead of failing. Finally, **narration comes first**: audio length sets scene length, so nothing is re-timed afterwards, and PDF pages that already have a text layer are read directly instead of through OCR.

Technology

Backend: Python, FastAPI, Pydantic, SQLite, PyTorch, ONNX Runtime, Transformers, sentence-transformers, FAISS, PyMuPDF. **Web:** React, TypeScript, Vite, Remotion (player and MP4 renderer with Chrome and ffmpeg), three.js, Mermaid. **Android:** Kotlin, Jetpack Compose, Room, WorkManager, Media3. **3D lessons:** a headless Blender pipeline. The interface follows the team's Figma design.

Impact

A student without reliable internet gets a patient teacher in their own language. A teacher can turn a textbook chapter into a narrated video in minutes. A school can run it on its own machines with no per-use cost and no data leaving the room. Wikipedia text is used under CC BY-SA 4.0 and each lesson cites the articles it drew on; the licences of every model and voice are listed in the README.

Run it

```
git clone https://github.com/signingoff-dubey/UNLOX-TEACHER.git
.\setup.ps1 -Gpu     # once, online
.\start.ps1          # http://127.0.0.1:8000
```

Requires Ollama with `qwen2.5:7b-instruct-q4_K_M`, `nomic-embed-text` and `llava:7b`. Complete documentation is in `Chanakya-Documentation.pdf`.